UNIVERSITY OF AUCKLAND
Waipapa Taumata Rau
NEW ZEALAND

SCIENCE

COMPSCI 714
Lecture 2: NN and DNN

Neural Networks and Deep Neural Network, what they are and how to train them.

Instructor: Thomas Lacombe

Learning outcomes

- Understand what is an artificial neuron, also called Perceptron.
- Understand what is a Neural Network (NN) and a Deep Neural Network (DNN).
- Understand how to train a DNN using gradient descent and backpropagation.

Disclaimer/Acknowledgment:

Some of the following slides are reusing and adapting some of the content created by Dan Klein and Pieter Abbeel for CS188 Intro to AI at UC Berkeley.

All CS188 materials are available at http://ai.berkeley.edu.

Some content is also reused from the CS230 Stanford course slides created by Andrew Ng, available at https://cs230.stanford.edu/syllabus/.

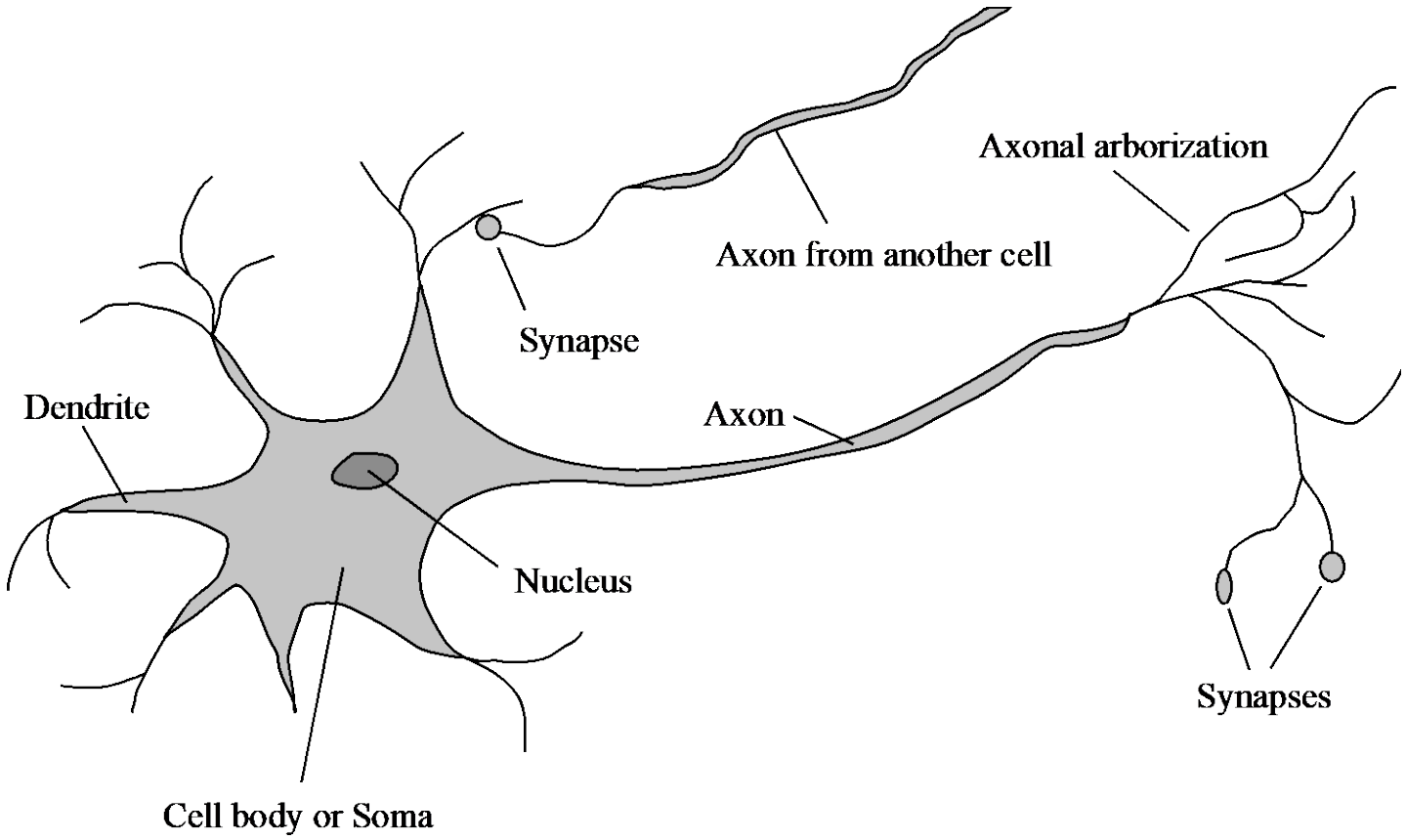
Extra resources

If you want to go further:

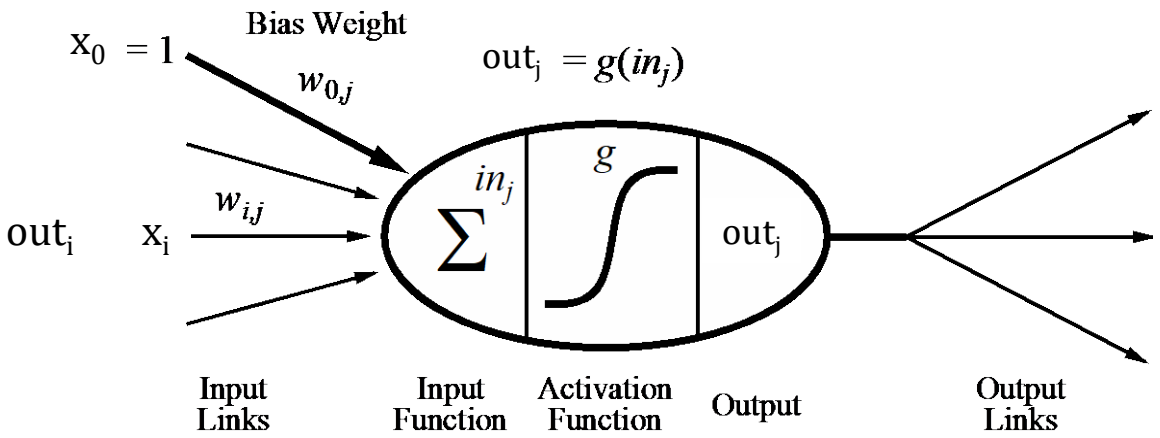
- *Hands-on Machine Learning with Scikit-Learn, Keras and Tensorflow*, Géron A.
  - Chapters 4 and 10

- *Deep Learning*, Goodfellow I. et al.
  - Part II Deep Networks
  - Part I Applied Math and Machine Learning Basics

Inspiration: Human neurons



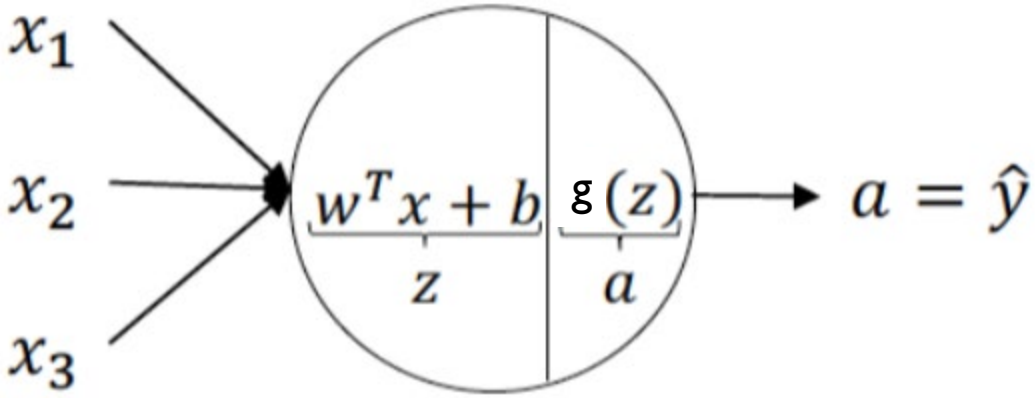
Simple model of a neuron (McCulloch & Pitts, 1943)



- Input x_i comes from the output of neuron i to this neuron j (or from "outside")
- Each input link has a ***weight*** $w_{i,j}$
- There is an additional fixed input x_0 with ***bias*** weight $w_{0,j}$ (or b_j)
- The total input is $in_j = \Sigma_i\, w_{i,j}\, x_i$ (or $\Sigma_i\, w_{i,j}\, x_i + b_j$)
- The output is $out_j = g(in_j) = g(\Sigma_i\, w_{i,j}\, x_i)$ (or $g(\Sigma_i\, w_{i,j}\, x_i + b_j)$)

→ Also called a **Perceptron**!

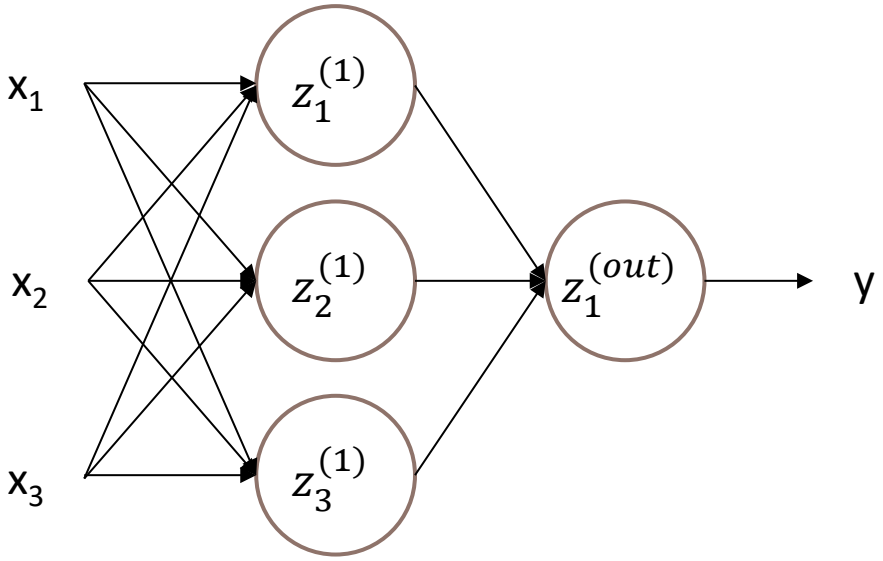
Linear algebra representation



$$z = w^T x + b$$

$$a = g(z)$$

Neural network or Multilayer Perceptron



Input layer Hidden layer Output layer

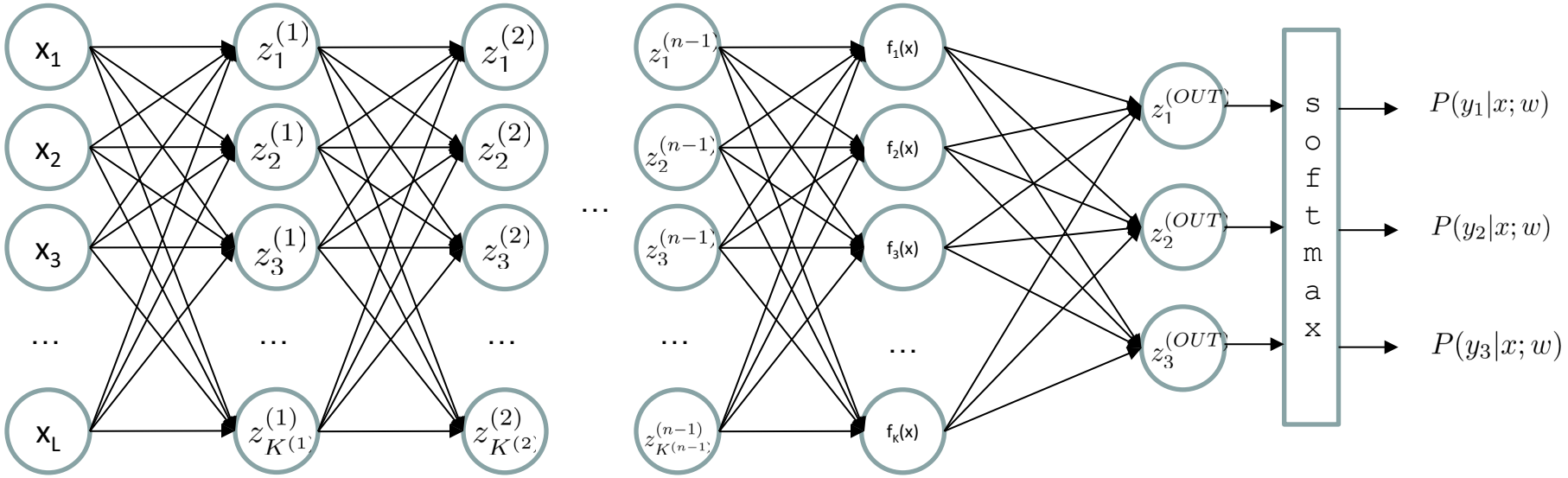
$in_z_i^{(k)}$ is the input of neuron i in layer k (after input function)

$out_z_i^{(k)}$ is the output of neuron i in layer k (after activation)

Deep Neural Network = Also learn the features!

- Deep NN = at least 2 hidden layers

Composite function of the inputs!

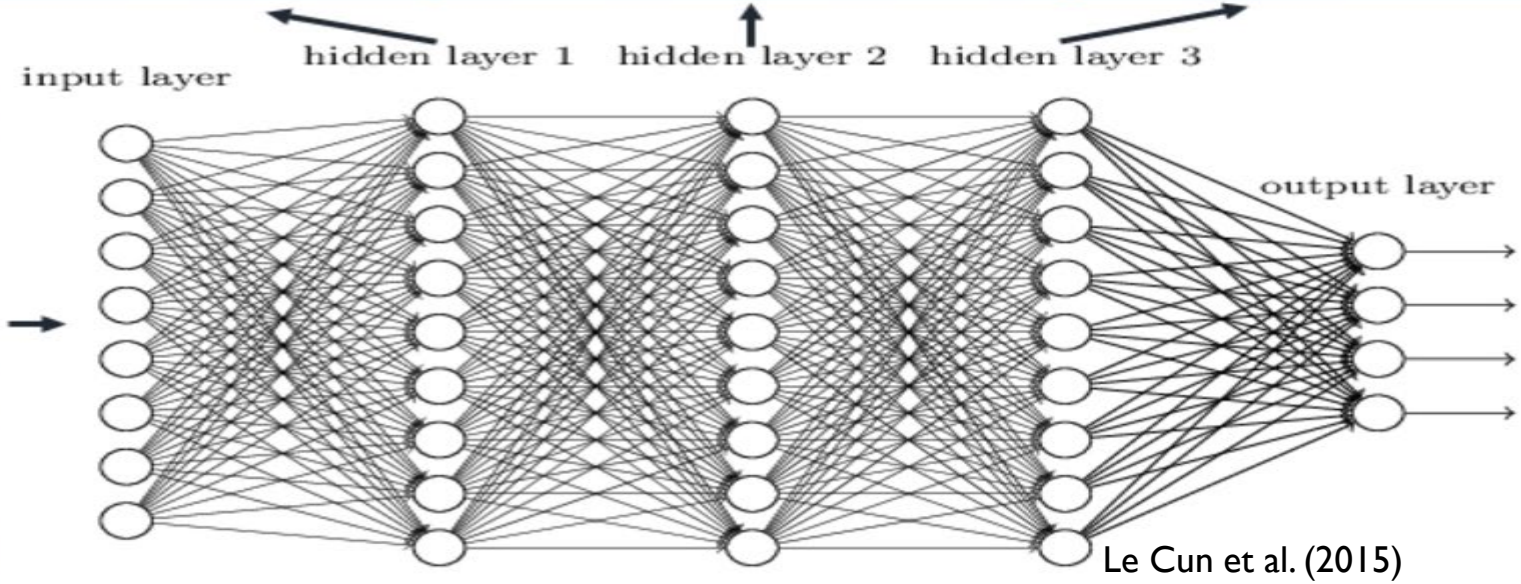


$$out_z_i^{(k)} = g\left(in_z_i^{(k)}\right) = g\left(\sum_j w_{i,j}^{(k-1,k)} out_z_j^{(k-1)}\right)$$

g = nonlinear activation function

Intuition about deep features representation

Deep neural networks learn hierarchical feature representations



Le Cun et al. (2015)
Raphael et al. (2019)

Neural Networks Properties

- **Theorem of the Universal Function Approximators**:

A two-layer neural network (one hidden layer) with a sufficient number of neurons can **approximate** any continuous function to any desired accuracy.

- Practical considerations:
  - Can be seen as learning the features.
  - Large number of neurons:
    - Danger of overfitting

Universal Function Approximation Theorem

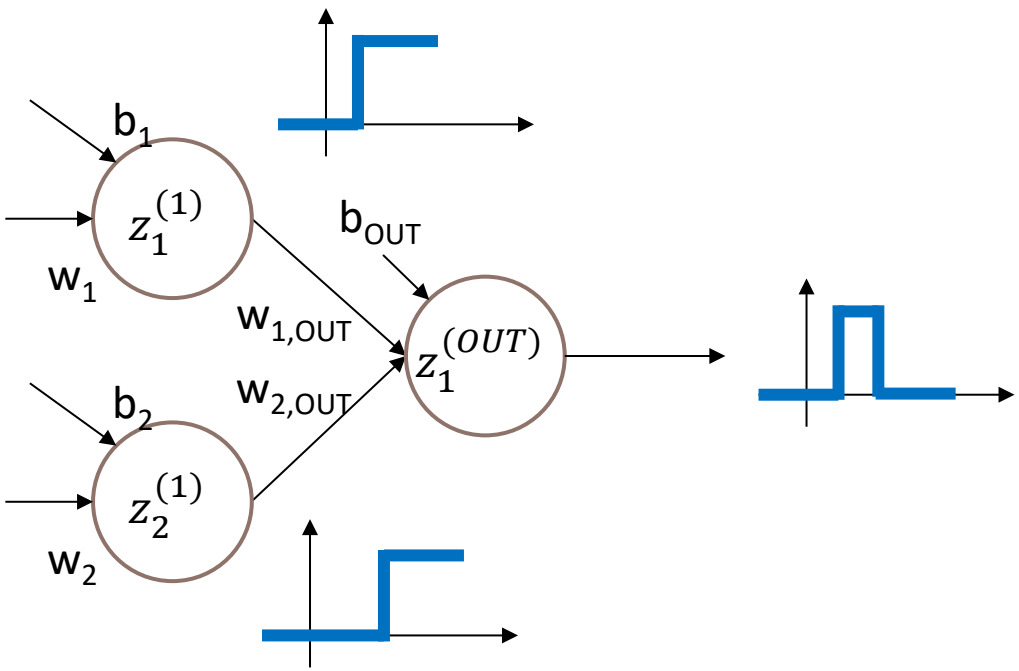
Hornik theorem 1: Whenever the activation function is *bounded and nonconstant*, then, for any finite measure μ, standard multilayer feedforward networks can approximate any function in $L^p(\mu)$ (the space of all functions on R^k such that $\int_{R^k} |f(x)|^p d\mu(x) < \infty$) arbitrarily well, provided that sufficiently many hidden units are available.

Hornik theorem 2: Whenever the activation function is *continuous, bounded and nonconstant*, then, for arbitrary compact subsets $X \subseteq R^k$, standard multilayer feedforward networks can approximate any continuous function on X arbitrarily well with respect to uniform distance, provided that sufficiently many hidden units are available.

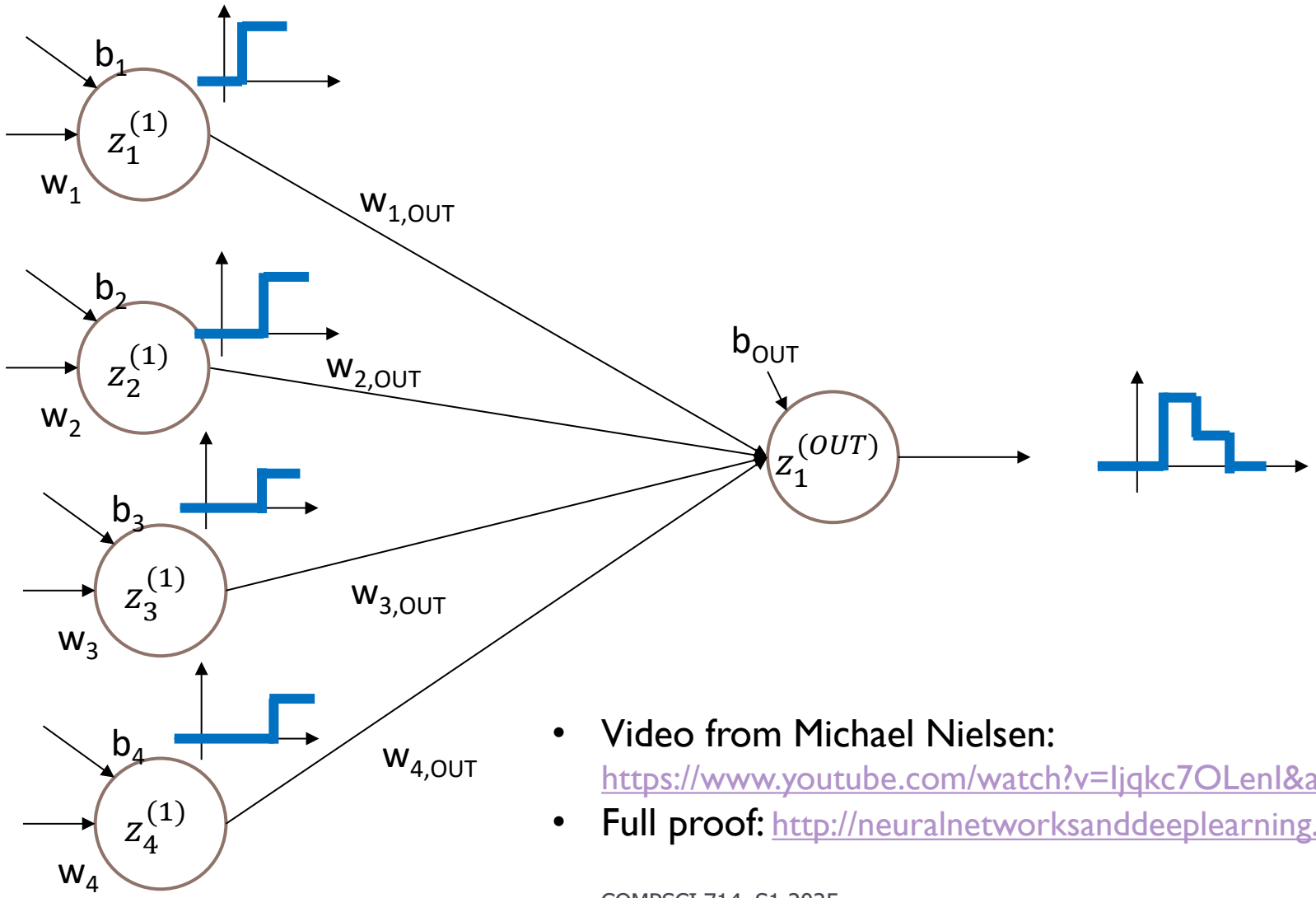
- In simple words: Given any continuous function f(x), if a 2-layer neural network has enough hidden units, then there is a choice of weights that allow it to closely approximate f(x).

Cybenko (1989) “Approximations by superpositions of sigmoidal functions”
Hornik (1991) “Approximation Capabilities of Multilayer Feedforward Networks”
Leshno and Schocken (1991) ”Multilayer Feedforward Networks with Non-Polynomial Activation Functions Can Approximate Any Function”

Intuition for the theorem



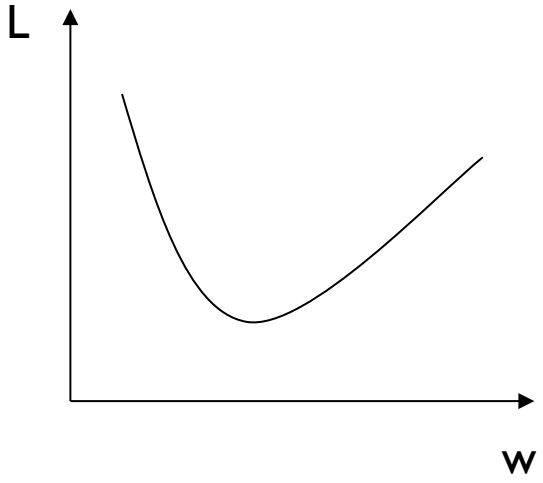
Intuition for the theorem



- Video from Michael Nielsen: https://www.youtube.com/watch?v=Ijqkc7OLenI&ab_channel=MichaelNielsen
- Full proof: http://neuralnetworksanddeeplearning.com/chap4.html

Learning a function is an optimisation problem

- Optimisation of the parameters (weights) to minimise a cost/loss/error function



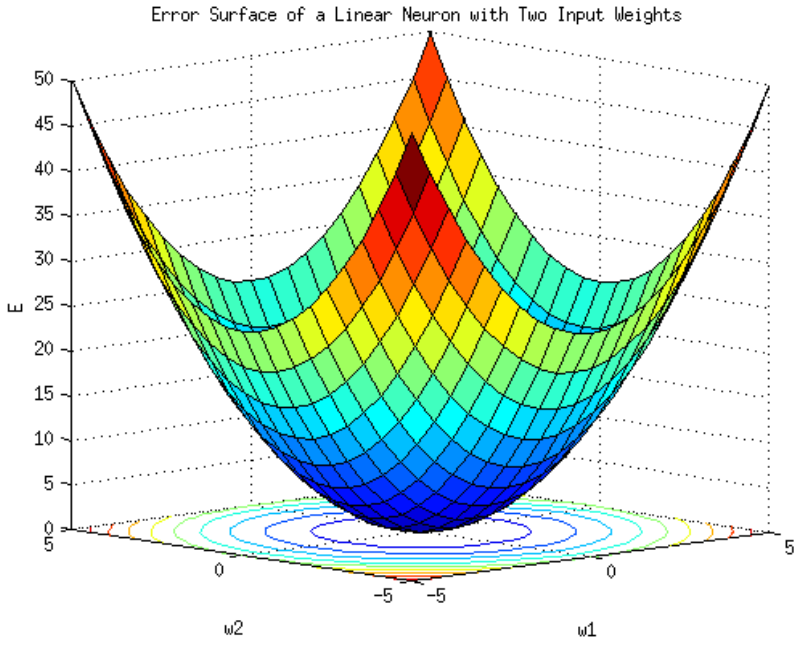


Figure source: https://srinivas-yeeda.medium.com/loss-functions-in-deep-learning-models-129866be93e

Gradient Descent

- Perform update in downhill direction for each coordinate.
- The steeper the slope (i.e. the higher the derivative) the bigger the step for that coordinate.
- E.g., consider: $g(w_1, w_2)$
  - Updates:

$$w_1 \leftarrow w_1 - \alpha * \frac{\partial g}{\partial w_1}(w_1, w_2)$$

$$w_2 \leftarrow w_2 - \alpha * \frac{\partial g}{\partial w_2}(w_1, w_2)$$

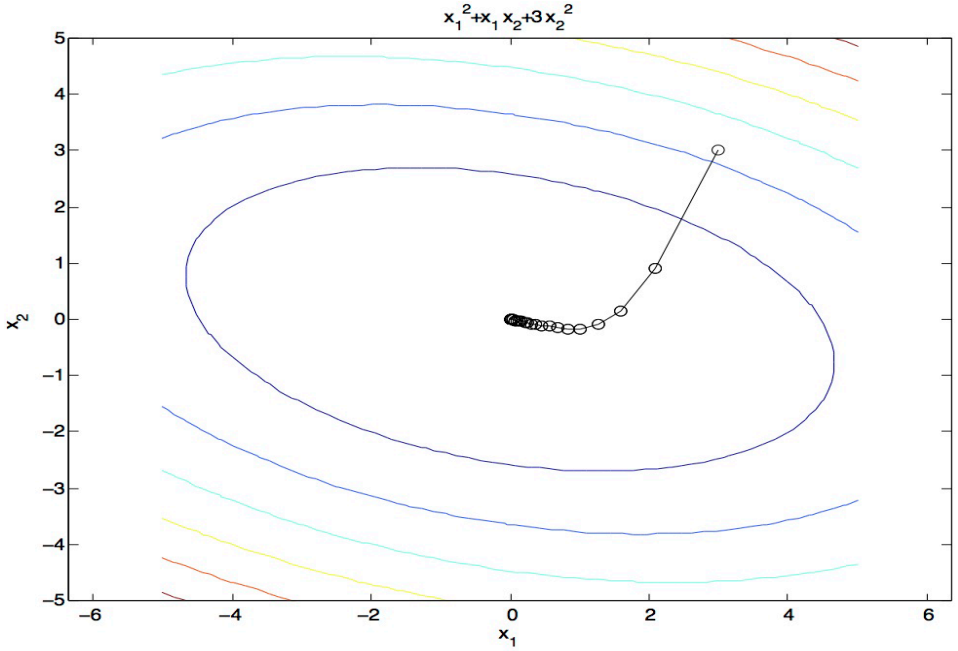
- Updates in vector notation:

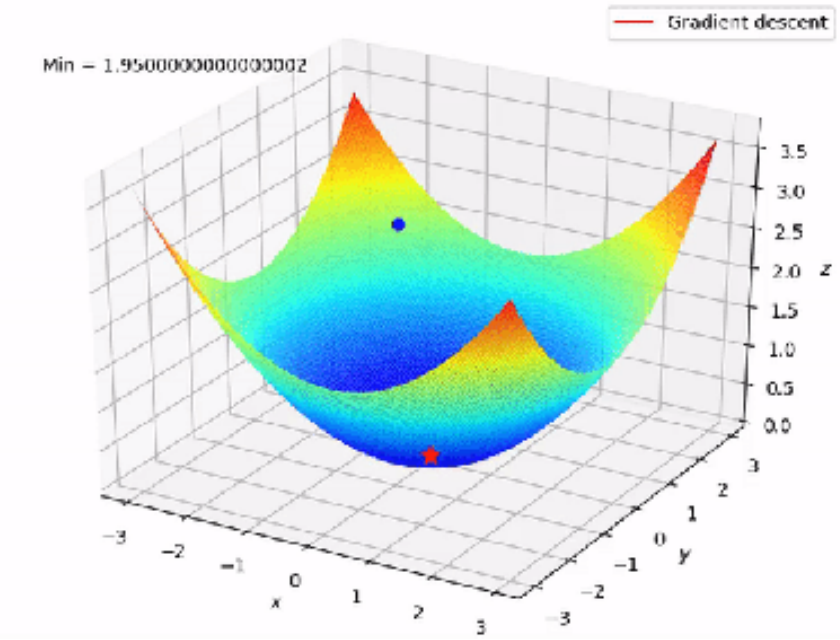
$$w \leftarrow w - \alpha * \nabla_w g(w)$$

with: $\nabla_w g(w) = \begin{bmatrix} \frac{\partial g}{\partial w_1}(w) \\ \frac{\partial g}{\partial w_2}(w) \end{bmatrix}$ **= gradient**

Steepest Descent

- Idea:
  - Start somewhere
  - Repeat: Take a step in the steepest descent direction





Figures source: Mathworks and https://suniljangirblog.wordpress.com/2018/12/03/the-outline-of-gradient-descent/

Steepest Direction

- Steepest Direction = direction of the gradient

$$\nabla g = \begin{bmatrix} \frac{\partial g}{\partial w_1} \\ \frac{\partial g}{\partial w_2} \\ \cdots \\ \frac{\partial g}{\partial w_n} \end{bmatrix}$$

Optimisation Procedure: Gradient Descent

- init w
- for iter = 1, 2, …

$$w \leftarrow w - \alpha * \nabla g(w)$$

- α : learning rate --- hyperparameter that needs to be chosen carefully

- If too high → chance to miss the optimum
- If too low → very long time

Gradient Descent / Stochastic GD / Mini-batch GD

- Gradient Descent (GD)/ Batch GD: updates weights based on loss over the full training data (batch size = dataset size)
  - → Smooth descent as using average gradient over training set (true gradient)
  - → Can be used for small datasets (max 2000 instances)

- Stochastic GD (SGD): updates weights based on loss over a randomly chosen instance of data (1 datapoint / iteration)
  - → More efficient than GD but fluctuates a lot

- Mini-batch GD: updates weights based on loss over a randomly chosen subset of data (>1 datapoint / iteration)
  - → More efficient than GD and less fluctuations than SGD
  - → Typical batch sizes: 64, 128, 256, 512, … (fit CPU/GPU memory)

Gradient Descent / Stochastic GD, Mini-batch GD

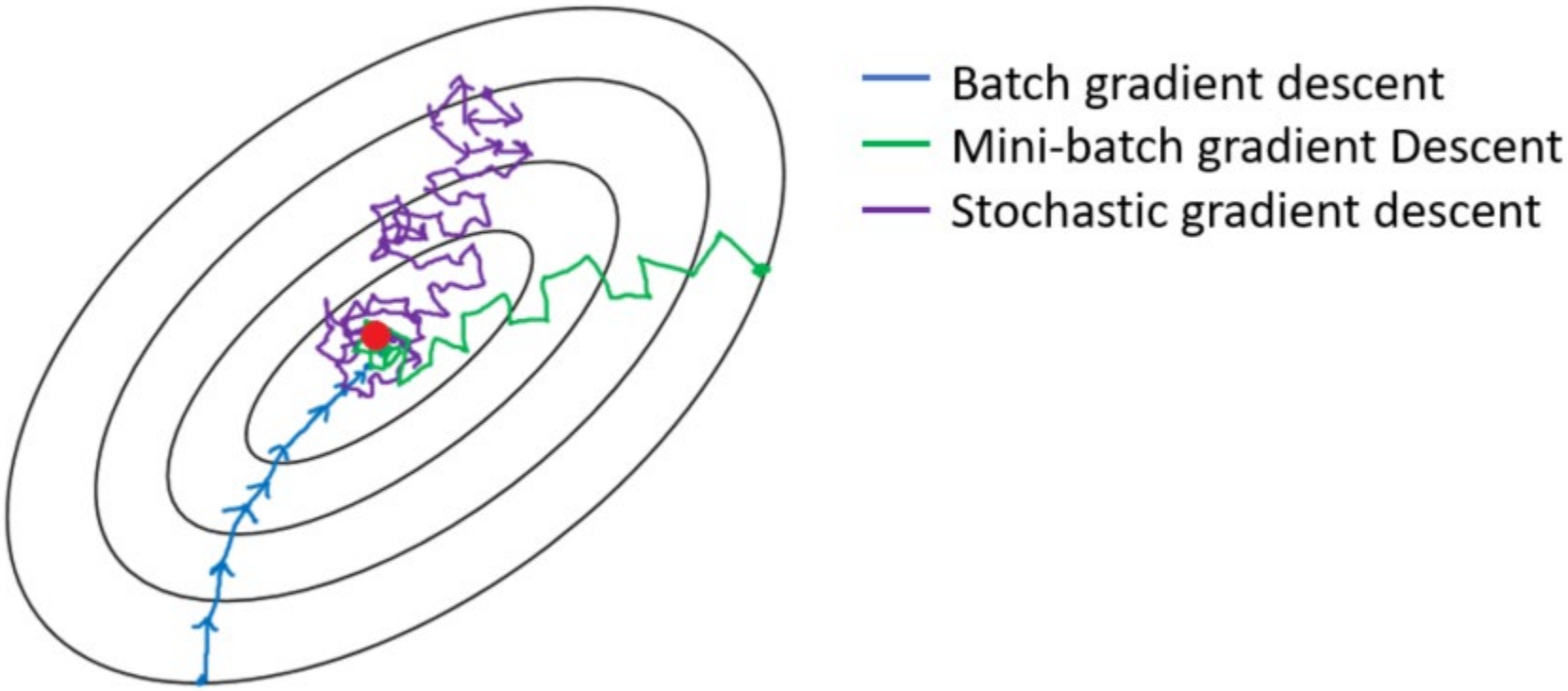


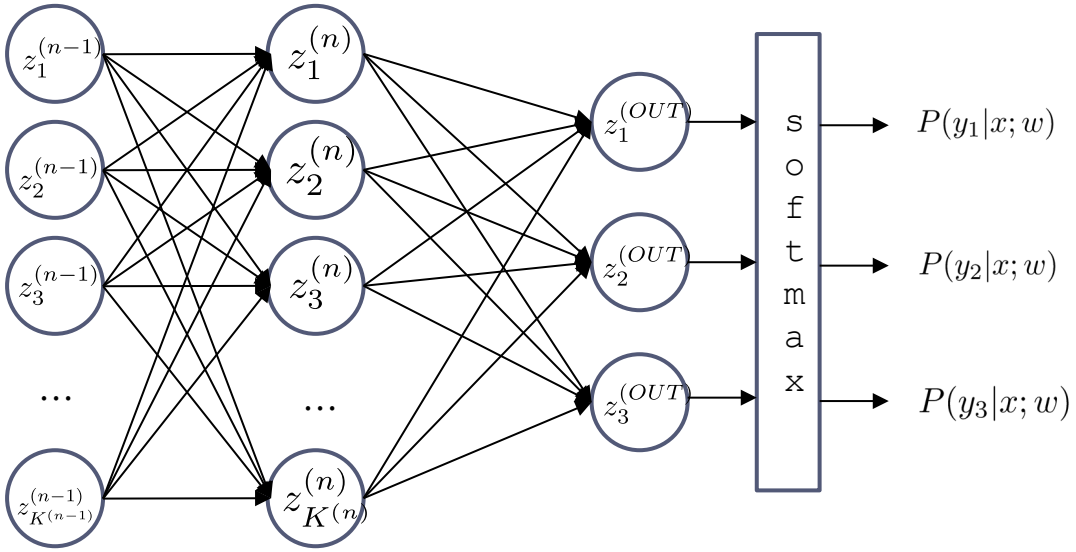
Figure source: https://medium.com/analytics-vidhya/gradient-descent-vs-stochastic-gd-vs-mini-batch-sgd-fbd3a2cb4ba4

Training a full network

- It is practical to apply gradient descent to update the output layer, given the gradient calculated from the output error/loss.

- But how to do this for other layers?

- This was solved in the 1980's by applying the backpropagation method (1960's) to training MLPs.



How about computing all the derivatives?

- Derivatives tables:

$$\frac{d}{dx}(a) = 0$$

$$\frac{d}{dx}(x) = 1$$

$$\frac{d}{dx}(au) = a\frac{du}{dx}$$

$$\frac{d}{dx}(u + v - w) = \frac{du}{dx} + \frac{dv}{dx} - \frac{dw}{dx}$$

$$\frac{d}{dx}(uv) = u\frac{dv}{dx} + v\frac{du}{dx}$$

$$\frac{d}{dx}\left(\frac{u}{v}\right) = \frac{1}{v}\frac{du}{dx} - \frac{u}{v^2}\frac{dv}{dx}$$

$$\frac{d}{dx}(u^n) = nu^{n-1}\frac{du}{dx}$$

$$\frac{d}{dx}(\sqrt{u}) = \frac{1}{2\sqrt{u}}\frac{du}{dx}$$

$$\frac{d}{dx}\left(\frac{1}{u}\right) = -\frac{1}{u^2}\frac{du}{dx}$$

$$\frac{d}{dx}\left(\frac{1}{u^n}\right) = -\frac{n}{u^{n+1}}\frac{du}{dx}$$

$$\frac{d}{dx}[f(u)] = \frac{d}{du}[f(u)]\frac{du}{dx}$$

$$\frac{d}{dx}[\ln u] = \frac{d}{dx}[\log_e u] = \frac{1}{u}\frac{du}{dx}$$

$$\frac{d}{dx}[\log_a u] = \log_a e\frac{1}{u}\frac{du}{dx}$$

$$\frac{d}{dx}e^u = e^u\frac{du}{dx}$$

$$\frac{d}{dx}a^u = a^u \ln a\frac{du}{dx}$$

$$\frac{d}{dx}(u^v) = vu^{v-1}\frac{du}{dx} + \ln u \;\; u^v\frac{dv}{dx}$$

$$\frac{d}{dx}\sin u = \cos u\frac{du}{dx}$$

$$\frac{d}{dx}\cos u = -\sin u\frac{du}{dx}$$

$$\frac{d}{dx}\tan u = \sec^2 u\frac{du}{dx}$$

$$\frac{d}{dx}\cot u = -\csc^2 u\frac{du}{dx}$$

$$\frac{d}{dx}\sec u = \sec u \tan u\frac{du}{dx}$$

$$\frac{d}{dx}\csc u = -\csc u \cot u\frac{du}{dx}$$

Image source: http://hyperphysics.phy-astr.gsu.edu/hbase/Math/derfunc.html

How about computing all the derivatives?

- But what if neural net function is not in the table?
  - No problem: CHAIN RULE:

$$out_z_i^{(k)} = g\left(\sum_j w_{i,j}^{(k-1,k)} out_z_j^{(k-1)}\right)$$

  - If $$f(x) = g(h(x))$$
  - Then $$f'(x) = g'(h(x))h'(x)$$
  - Derivatives can be computed by following well-defined procedures.

Automatic Differentiation

- Automatic differentiation software
  - e.g. Theano, TensorFlow, PyTorch, Chainer.
  - Only need to program the function g(x,y,w).
  - Can automatically compute all derivatives w.r.t. all entries in w.
  - This is typically done by caching info during forward computation pass of f, and then doing a backward pass = “backpropagation”.
  - Autodiff / Backpropagation can often be done at computational cost comparable to the forward pass.

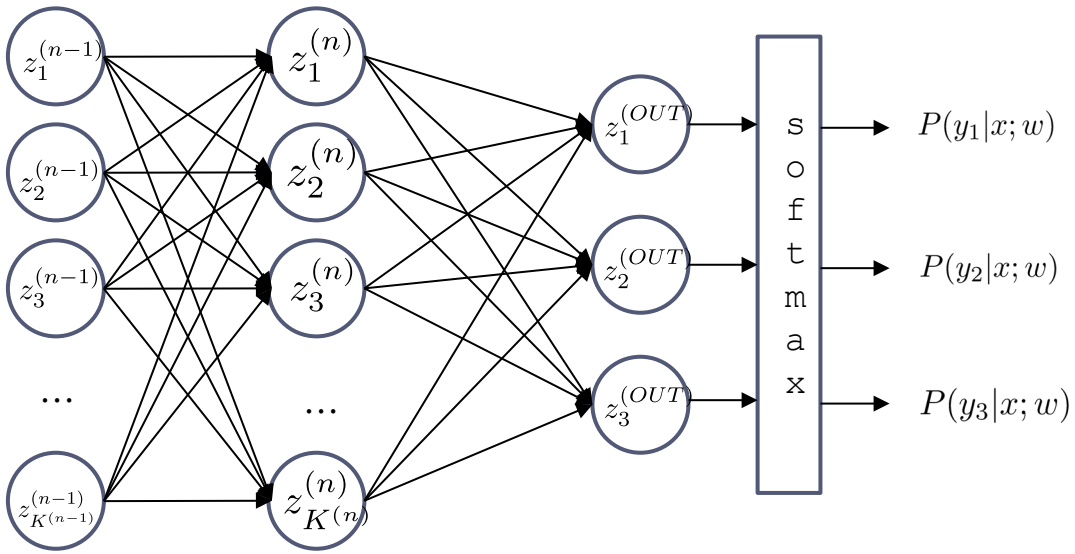
- Need to know this exists and understand the process.

- If you want more details, you can read Appendix B of the “Hands on ML” book.

Training a Network

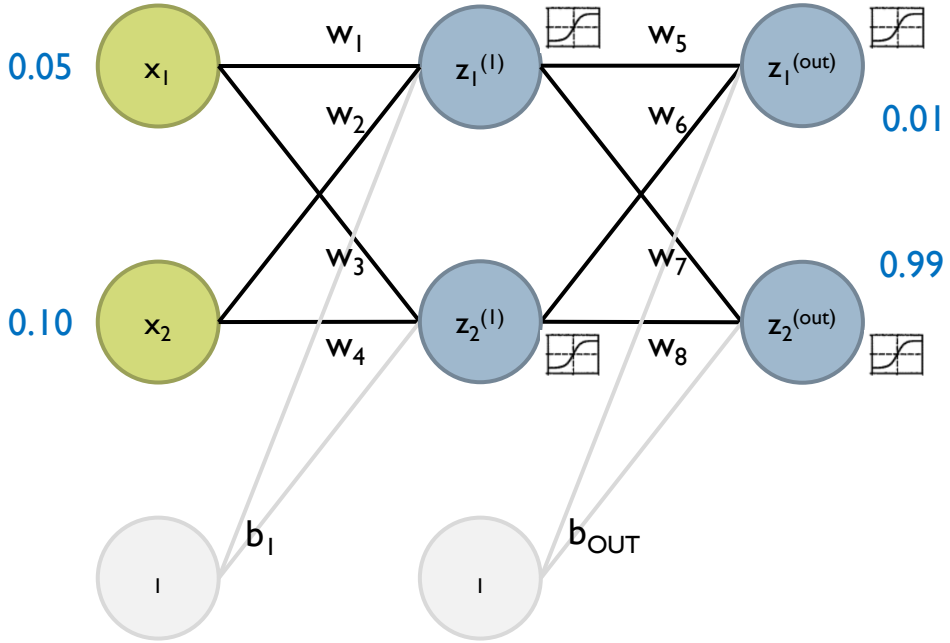
Keywords:

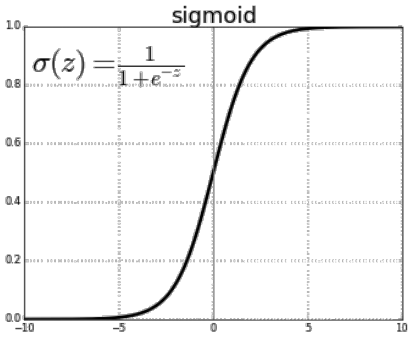
- Forward
- Backward
- Gradient
- Backprop



Dan Klein and Pieter Abbeel

Back Propagation in a NN

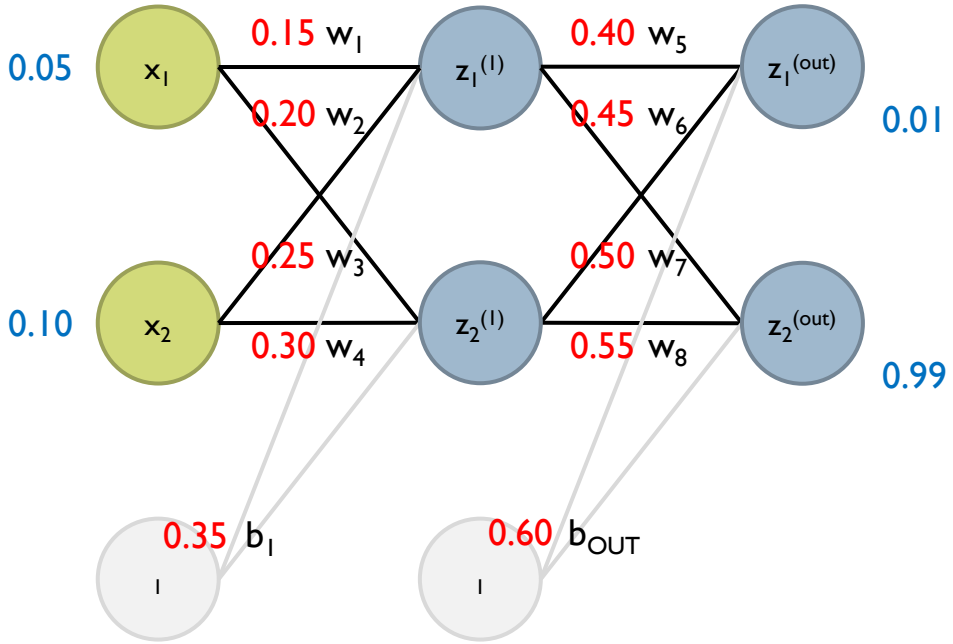




Example source: https://mattmazur.com/2015/03/17/a-step-by-step-backpropagation-example/

Back Propagation in a NN

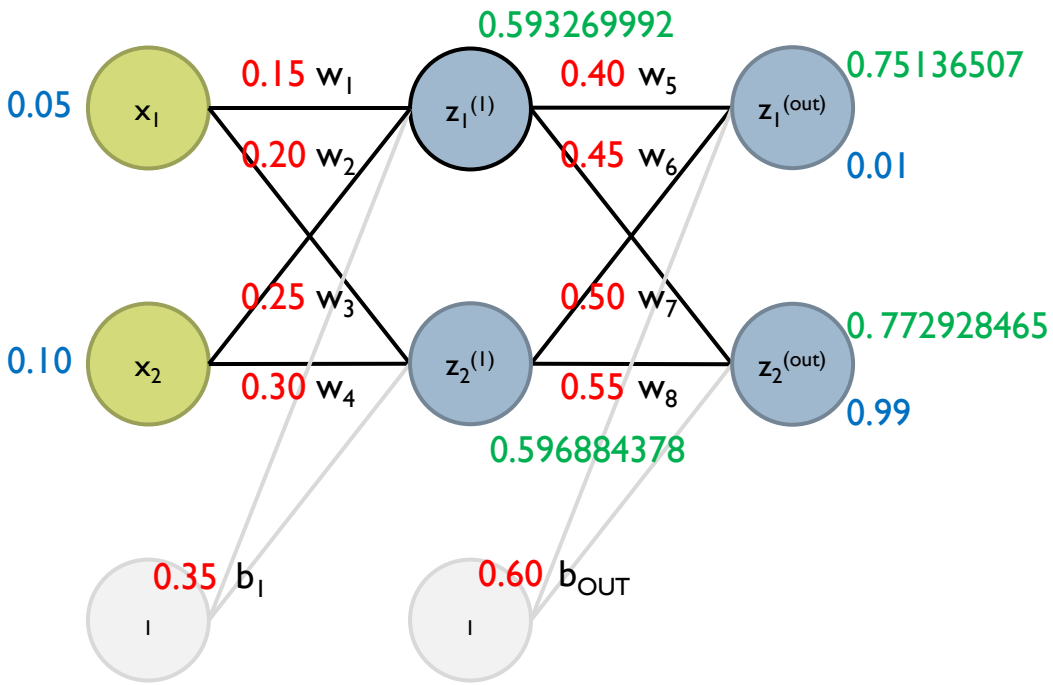
- Initialisation of the weights



Example source: https://mattmazur.com/2015/03/17/a-step-by-step-backpropagation-example/

Backpropagation in a NN

- Forward propagation



$$\text{out_}z_1^{(1)} = S(\overbrace{w_1 * x_1 + w_2 * x_2 + b_1 * 1}^{\text{in_}z_1^{(1)}})$$
$$= S(0.15 * 0.05 + 0.20 * 0.10 + 0.35 * 1)$$
$$= S(0.3775)$$
$$= 0.593269992$$

$$\text{out_}z_2^{(1)} = S(\overbrace{w_3 * x_1 + w_4 * x_2 + b_1 * 1}^{\text{in_}z_2^{(2)}})$$
$$= S(0.25 * 0.05 + 0.30 * 0.10 + 0.35 * 1)$$
$$= 0.596884378$$

$$\text{out_}z_1^{(out)} = S(\overbrace{w_5 * \text{out_}z_1^{(1)} + w_6 * \text{out_}z_2^{(1)} + b_{OUT} * 1}^{\text{in_}z_1^{(out)}})$$
$$= 0.75136507$$

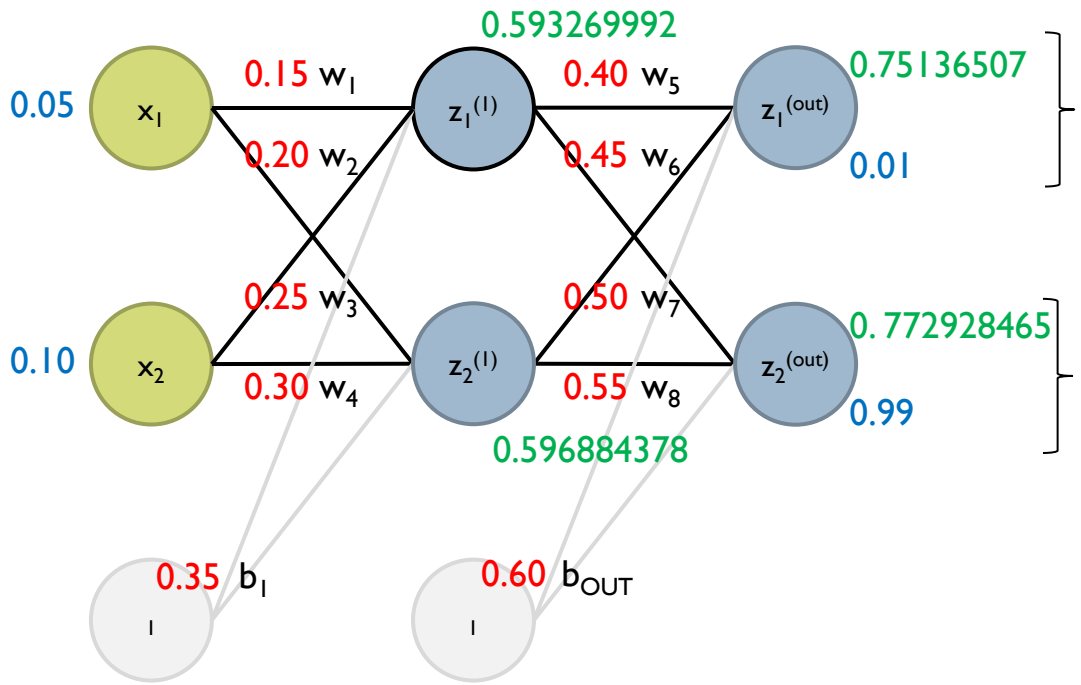
$$\text{out_}z_1^{(out)} = S(\overbrace{w_7 * \text{out_}z_1^{(1)} + w_8 * \text{out_}z_2^{(1)} + b_{OUT} * 1}^{\text{in_}z_2^{(out)}})$$
$$= 0.772928465$$

Example source: https://mattmazur.com/2015/03/17/a-step-by-step-backpropagation-example/

Backpropagation in a NN

- Calculating the total error

$$E(y, y') = \frac{1}{2}(y - y')^2$$



$$E_{z_1}{}^{(out)} = \frac{1}{2}(0.01 - 0.75136507)^2$$
$$= 0.274811083$$

$$E_{z_2}{}^{(out)} = \frac{1}{2}(0.99 - 0.772928465)^2$$
$$= 0.023560026$$

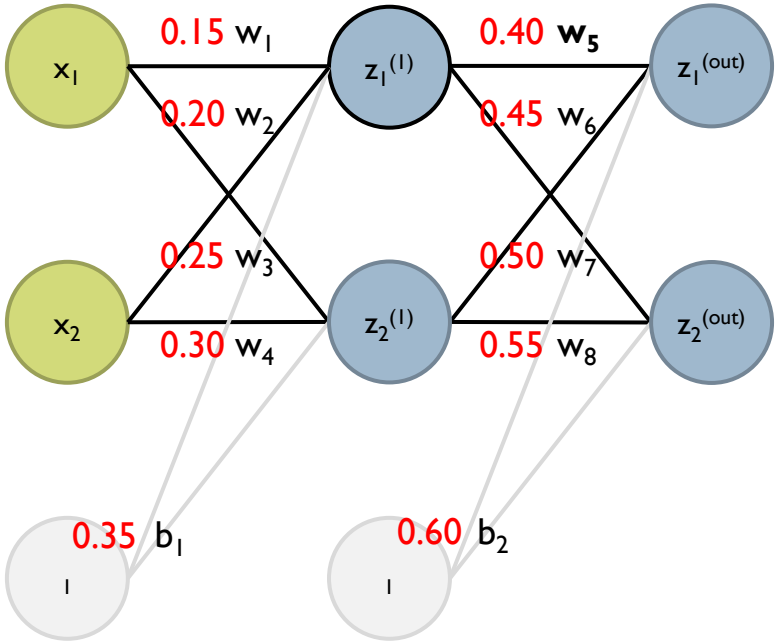
$$E_{tot}{}^{(out)} = E_{z_1}{}^{(out)} + E_{z_2}{}^{(out)}$$
$$= 0.298371109$$

What type of GD?

Example source: https://mattmazur.com/2015/03/17/a-step-by-step-backpropagation-example/

Backpropagation in a NN

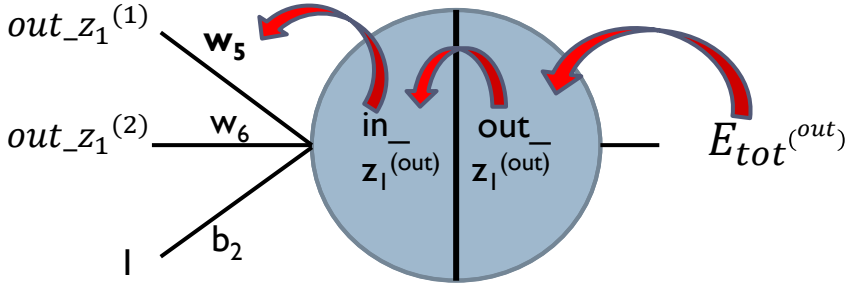
- Back propagation (output layer)



$\mathbf{w_5}$ influence on $E_{tot}^{(out)}$? $\dfrac{\partial E_{tot}^{(out)}}{\partial \mathbf{w}_5}$

Chain rule:

$$\frac{\partial E_{tot}^{(out)}}{\partial \mathbf{w}_5} = \frac{\partial E_{tot}^{(out)}}{\partial out_z_1^{(out)}} * \frac{\partial out_z_1^{(out)}}{\partial in_z_1^{(out)}} * \frac{\partial in_z_1^{(out)}}{\partial \mathbf{w}_5}$$



Example source: https://mattmazur.com/2015/03/17/a-step-by-step-backpropagation-example/

Backpropagation in a NN

- Back propagation (output layer)

$$\frac{\partial E_{tot}{}^{(out)}}{\partial \mathbf{w}_5} = \frac{\boldsymbol{\partial E_{tot}{}^{(out)}}}{\boldsymbol{\partial out_z_1{}^{(out)}}} * \frac{\partial out_z_1{}^{(out)}}{\partial in_z_1{}^{(out)}} * \frac{\partial in_z_1{}^{(out)}}{\partial \mathbf{w}_5}$$

$$\begin{aligned} E_{tot}{}^{(out)} &= E_{z_1}{}^{(out)} + E_{z_2}{}^{(out)} \\ &= \frac{1}{2}\left(target_{z_1(out)} - out_z_1{}^{(out)}\right)^2 + \frac{1}{2}\left(target_{z_2(out)} - out_z_2{}^{(out)}\right)^2 \end{aligned}$$

$$\begin{aligned} \frac{\boldsymbol{\partial E_{tot}{}^{(out)}}}{\boldsymbol{\partial out_z_1{}^{(out)}}} &= 2 * \frac{1}{2}\left(target_{z_1(out)} - out_{z_1(out)}\right)^{2-1} * -1 + 0 \\ &= out_{z_1(out)} - target_{z_1(out)} \\ &= 0.75136507 - 0.01 \\ &= 0.74136507 \end{aligned}$$

Example source: https://mattmazur.com/2015/03/17/a-step-by-step-backpropagation-example/

Backpropagation in a NN

- Back propagation (output layer)

$$\frac{\partial E_{tot}{}^{(out)}}{\partial \mathbf{w}_5} = \frac{\partial E_{tot}{}^{(out)}}{\partial out_z_1{}^{(out)}} * \frac{\partial out_z_1{}^{(out)}}{\partial in_z_1{}^{(out)}} * \frac{\partial in_z_1{}^{(out)}}{\partial \mathbf{w}_5}$$

$$out_{z_1}{}^{(out)} = \frac{1}{1+e^{-in_z_1{}^{(out)}}}$$ (activation = sigmoid function)

Sigmoid → g'(x) = g(x)(1-g(x))

$$\frac{\partial out_z_1{}^{(out)}}{\partial in_z_1{}^{(out)}} = out_z_1{}^{(out)} * (1 - out_z_1{}^{(out)})$$
$$= 0.75136507 * (1 - 0.75136507)$$
$$= 0.186815602$$

Example source: https://mattmazur.com/2015/03/17/a-step-by-step-backpropagation-example/

Backpropagation in a NN

- Back propagation (output layer)

$$\frac{\partial E_{tot}^{(out)}}{\partial \mathbf{w_5}} = \frac{\partial E_{tot}^{(out)}}{\partial out_z_1^{(out)}} * \frac{\partial out_z_1^{(out)}}{\partial in_z_1^{(out)}} * \frac{\partial in_z_1^{(out)}}{\partial \mathbf{w_5}}$$

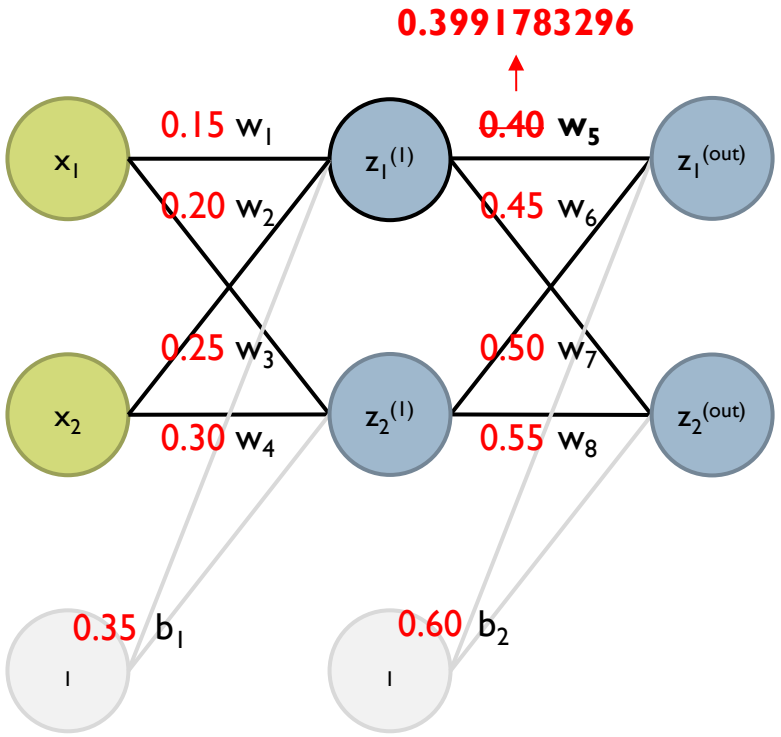
$$\mathrm{in_z_1}^{(\mathrm{out})} = \mathrm{w_5} * \mathrm{out_z_1}^{(1)} + \mathrm{w_6} * \mathrm{out_z_2}^{(1)} + \mathrm{b_2} * 1$$

$$\begin{aligned}\frac{\partial in_z_1^{(out)}}{\partial \mathbf{w_5}} &= \mathrm{out_z_1}^{(1)} + 0 + 0 \\ &= \mathrm{out_z_1}^{(1)} \\ &= 0.593269992\end{aligned}$$

Example source: https://mattmazur.com/2015/03/17/a-step-by-step-backpropagation-example/

Backpropagation in a NN

- Back propagation (output layer)



$$\frac{\partial E_{tot}^{(out)}}{\partial \mathbf{w_5}} = \frac{\partial E_{tot}^{(out)}}{\partial out_z_1^{(out)}} * \frac{\partial \mathrm{out_z_1}^{(\mathrm{out})}}{\partial \mathrm{in_z_1}^{(\mathrm{out})}} * \frac{\partial in_z_1^{(out)}}{\partial \mathbf{w_5}}$$

$$= 0.74136507 * 0.186815602 * 0.593269992$$
$$= 0.082167041$$

Gradient descent to decrease the error
(with learning rate $\alpha = 0.01$) :

$$\mathrm{w_5} \leftarrow \mathrm{w_5} - \alpha * \frac{\partial E_{tot}^{(out)}}{\partial \mathbf{w_5}}$$

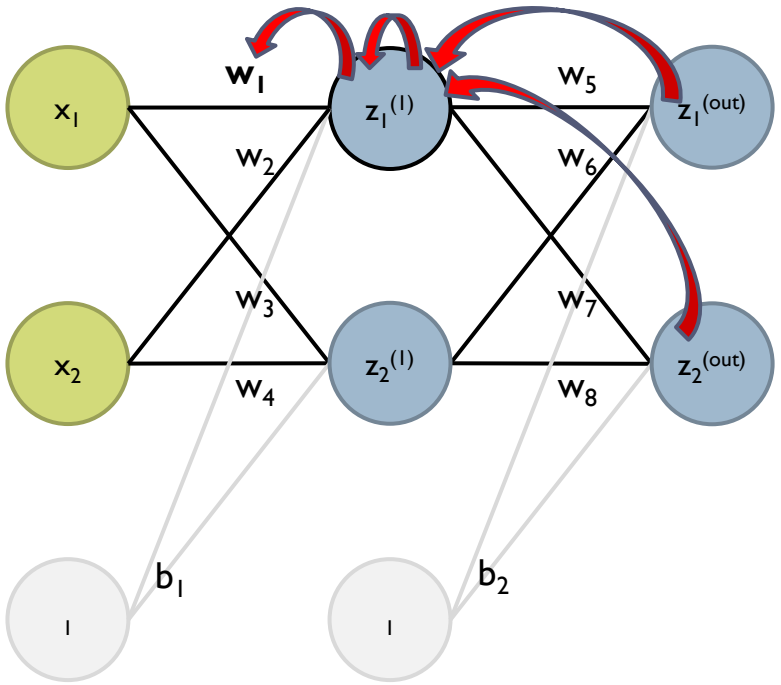
$$\mathrm{w_5} \leftarrow 0.40 - 0.01 * 0.082167041$$

$$\mathrm{w_5} \leftarrow 0.3991783296$$

Example source: https://mattmazur.com/2015/03/17/a-step-by-step-backpropagation-example/

Backpropagation in a NN

- Back propagation



$$\frac{\partial E_{tot}^{(out)}}{\partial \mathbf{w}_1} = \underbrace{\frac{\partial E_{tot}^{(out)}}{\partial out_z_1^{(1)}}} * \frac{\partial \text{out_z}_1^{(1)}}{\partial \text{in_z}_1^{(1)}} * \frac{\partial in_z_1^{(1)}}{\partial \mathbf{w}_1}$$

$$\frac{\partial E_{tot}^{(out)}}{\partial out_z_1^{(1)}} = \frac{\partial E_{z_1}^{(out)}}{\partial out_z_1^{(1)}} + \frac{\partial E_{z_2}^{(out)}}{\partial out_z_1^{(1)}}$$

Example source: https://mattmazur.com/2015/03/17/a-step-by-step-backpropagation-example/

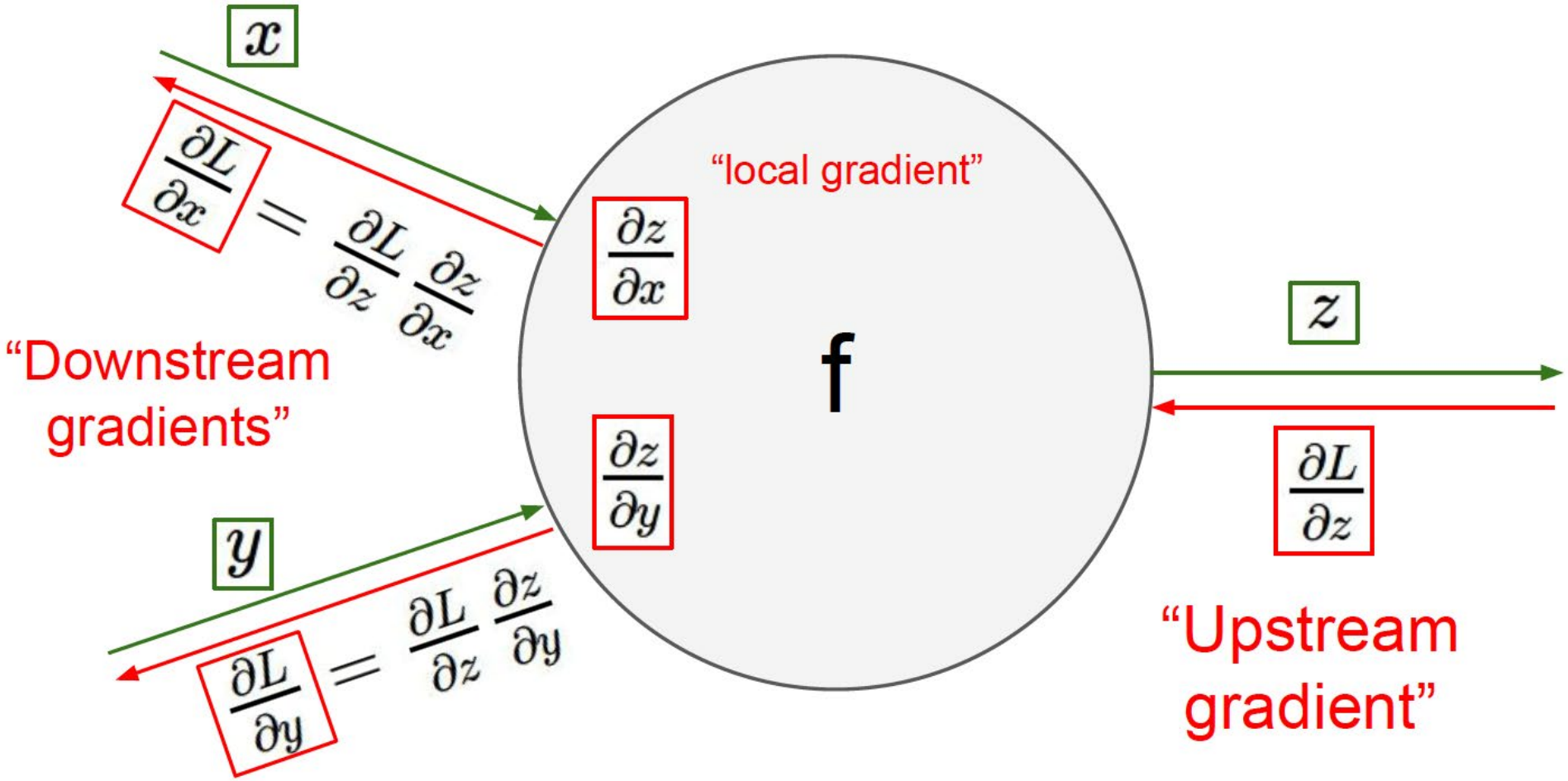


Figure source: CS188 Intro to AI at UC Berkeley

Conclusion

IT'S FRIDAY NIGHT, ALL
MY FRIENDS ARE AT THE PARTY
BUT, I'M HERE TRYING TO
FIGURE OUT BACKPROPAGATION
imgflip.com